

Solution Architecture

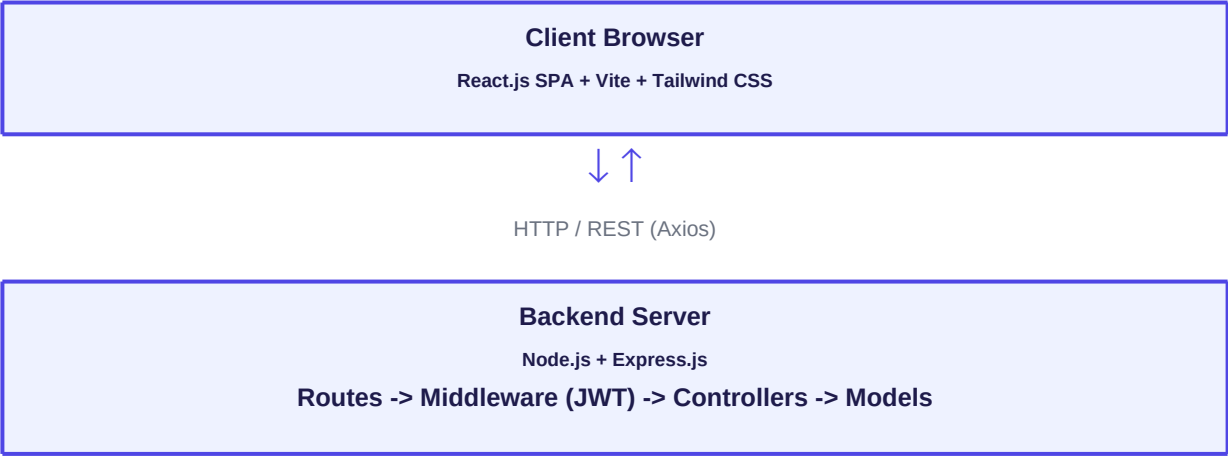
Phase 2: Project Design Phase

shopEZ E-commerce Marketplace | MERN Stack | June 2026

1. Architecture Pattern: Model-View-Controller (MVC)

shopEZ enforces strict decoupling of client-server application flows using the Model-View-Controller (MVC) design pattern mapped across the full MERN codebase.

2. System Architecture Diagram





Read / Write Models



Upload / Serve Media

MongoDB Atlas

Mongoose ODM

Local Uploads (Multer)

Product Image Assets

3. MVC Layer Breakdown

View Layer - Frontend (React.js + Vite)

- **Technology:** React.js 19.x + Vite 8.x + Tailwind CSS 3.x
- **Role:** Serves the dynamic Single Page Application (SPA) to the browser.
- **Context State Providers:** AuthContext (user session), CartContext (persistent cart items), ToastContext (global notifications).
- **Product Views:** Grid search lists with sliders, related product lists, and review feeds.
- **Checkout Wizard:** Handles shipping location forms, coupon validation flags, and Stripe input cards.

Controller Layer - Backend (Node.js + Express.js)

- **Technology:** Node.js 18+ + Express.js 4.x
- **Role:** Processes incoming RESTful API routes, validates payload bodies, and serves structured JSON responses.
- **authController.js:** Coordinates registrations, hashes passwords via Bcrypt, and compiles addresses/wishlists.
- **productController.js:** Formulates catalog search queries, aggregates ratings, and manages review logs.
- **orderController.js:** Coordinates stock adjustments, processes mock payments, and updates fulfillment timelines.
- **couponController.js:** Evaluates coupon expiration dates, active toggles, and discount computations.

Model Layer - Database (MongoDB + Mongoose)

- **Technology:** MongoDB 8.x + Mongoose ODM
- **Role:** Houses collections representing application state with schema validation.
- **User.js:** User profiles (name, email, role, wishlist array, saved shipping addresses array).

- **Product.js:** Inventory listings (title, description, price, stock, brand, category, rating, reviews list).
- **Order.js:** Financial and tracking items (purchased items, shipping, tax, total, payment status, timeline).
- **Coupon.js:** Discount entries (code, percentage/fixed value, active status, expiry dates).

4. Third-Party Integrations

Service	Purpose	Integration Method
Multer	Product image upload and local disk storage	Express middleware on POST /api/upload route; files saved to /backend/uploads/
Mock Stripe	Payment intent simulation for checkout flow	POST /api/payment/create-intent returns mock client_secret for Stripe Elements UI
JWT (jsonwebtoken)	Stateless user session authentication	Token signed on login; verified via authMiddleware.js on every protected route
Bcrypt.js	Password hashing before MongoDB persistence	bcryptjs.hash() with salt round factor of 10 on registration
mongodb-memory-server	In-memory MongoDB fallback for development	Auto-started by seeder.js if MONGO_URI connection fails
Chart.js	Admin analytics visualizations	react-chartjs-2 wrapper rendering bar/doughnut/line charts on AdminDashboard.jsx

5. API Request Flow

Step	From	Action	To
------	------	--------	----

1	Client Browser	REST API Requests (Axios)	Express Server
2	Express Server	Route Guarding	JWT Auth Middleware
3	Auth Middleware	Query Validation	Express Server
4	Express Server	Read/Write Models	MongoDB Atlas (Mongoose)
5	Express Server	Upload Media	Local Multer /uploads
6	Client Browser	Payment Intents	Mock Stripe Endpoint